

Wyatt Avilla

(408) 506-2189 | wyatt@linux.com | github.com/wyatt-avilla | linkedin.com/in/wyatt-avilla | wyattavilla.dev

EDUCATION

San José State University

Major: M.S Software Engineering, specializing in networking software

Relevant Courses: Database Systems, Operating System Design, Computer Network Design

University of California, Santa Cruz

Major: B.S. Cognitive Science, specializing in AI & HCI

Minor: Computer Science

Relevant Courses: Data Structures & Algorithms, Object Oriented Programming, Parallel Programming, Computer System Design, Artificial Intelligence

Aug 2025 - May 2027

CGPA: 4.0 Transcript [↗](#)

Sept 2021 - June 2025

CGPA: 3.9 Transcript [↗](#)

SKILLS

Programming Languages: Python, Rust, C, C++, SQL, Nix, Lua, Shell

Backend & APIs: FastAPI, Axum, WebSockets, REST, SQLAlchemy, Alembic, Pydantic, asyncio, asyncpg, Uvicorn, Tokio

Databases: PostgreSQL, SQLite, InfluxDB, MongoDB, MariaDB, MySQL

AI & ML: LangChain, LLM Evaluation, Hugging Face, OpenAI API, Ollama, Tool Calling

Infrastructure & DevOps: NixOS, Nix Flakes, systemd, nginx, Docker, GitHub Actions, Prometheus, CI/CD

Concepts: Concurrency, Async Programming, WebSocket Protocols, Distributed Systems, Networking, IoT, Embedded Systems

WORK EXPERIENCE

Member of the Technical Staff

Oct 2025 - Present

Circuit Breaker Labs

- Built Circuit Breaker Labs' FastAPI evaluation platform for AI safety red-teaming, owning typed REST and WebSocket endpoints for single-turn and multi-turn LLM evaluation workflows
- Designed WebSocket evaluation flows with typed protocol envelopes, protocol-version validation, progress notifications, completion-request routing, and close-code error mapping, with database-backed API key authentication and monthly quota enforcement shared across REST and WebSocket handlers
- Implemented model-provider call tracking across OpenAI, OpenRouter, and WebSocket client providers, capturing prompt contents, token counts, and errors to power precise API expenditure calculation and link generated tests and evaluated responses back to underlying model calls
- Designed and migrated PostgreSQL schemas for users, API keys, test cases, generation records, test results, provider-call logs, and quotas using SQLAlchemy async and Alembic
- Packaged and deployed the API with Nix flakes, uv2nix, a NixOS service module, systemd, PostgreSQL, agenix-managed secrets, nginx TLS/WebSocket proxying, and Prometheus/Alertmanager alerting, reducing build size from 17GB to 1GB and build time from 5+ hours to under a minute by removing CUDA dependencies and pinning Nix inputs
- Refactored evaluation and provider functions to errors-as-values, enabling per-test-case error reporting, and debugged production-only request hanging caused by HTTP client timeout and resource-leak issues only surfacing after extended server uptime
- Established CI/CD across Python and Nix codebases with GitHub Actions workflows for Ruff, strict Mypy, pytest coverage, Nix builds, statix, and nixfmt
- Built the public Rust cbl CLI client for the evaluation platform, implementing async WebSocket orchestration, multi-provider abstraction, and cross-platform release automation (see Projects)

Backend Developer Intern

Sept 2024 - Dec 2024

Lillup

- Built a FastAPI backend with LangChain integration for LLM tool calling, enabling the model to dynamically query user data through a RESTful endpoint that processed multi-turn conversations with tool execution
- Designed and implemented custom tool definitions with automated response parsing, handling JSON deserialization of LLM outputs into function arguments and returning structured responses
- Extended Markdown syntax using markdown-it-py and regex parsing to support structured metadata (tags, progress indicators, due dates) for improved LLM context in time-sensitive applications
- Established code quality standards with static typing (Mypy), Ruff for formatting/linting, and automated CI/CD pipeline through GitHub Actions

Programming Course Developer & Tutor

July 2024 - June 2025

University of California, Santa Cruz

- Independently migrated 4 programming assignments and 12 assessments (~2,000 lines) from C++ to Python, ensuring 100% test compatibility across 1,600+ test cases while implementing modern Python idioms including static typing, generics, comprehensions, and lazy evaluation
- Led group tutoring sessions for data structures and algorithms, focusing on problem-solving strategies for technical interview preparation

PROJECTS

Circuit Breaker Labs CLI [↗](#)

January 2026 - Present

Built and shipped cbl, the public Rust CLI for Circuit Breaker Labs' AI safety evaluation platform, connecting to a deployed FastAPI service over WebSockets with typed protocol envelopes, API-key authentication, and version negotiation. Drove evaluations through an async engine built on tokio::select! and JoinSet, with model providers abstracted behind a shared trait supporting OpenAI, Ollama, and Rhai-scripted custom integrations. Shipped Ratatui inline progress displays, 79 unit and integration tests, and cargo-dist releases targeting Apple Silicon, Intel macOS, Linux musl, and Windows MSVC.

Discord Bot with LLM Tool-Calling Integration [↗](#)

August 2025

Built a Discord bot in Rust enabling Anthropic's Claude to take actions in Discord servers through tool calling, featuring image analysis, message reactions, and configurable interaction patterns with per-server configuration management and probabilistic autonomous responses. Exposed as a NixOS service and packaged with Nix flakes.

Type-Safe REST API with ESP32 Client Integration [↗](#)

June 2025

Built a production-ready REST API in Rust using Actix Web to monitor systemd service statuses, with shared type definitions ensuring compile-time safety between server and ESP32 client firmware. Implemented asynchronous message-passing architecture using Embassy for low-power consumption, packaged as a configurable NixOS service with CLI interface for real-time system monitoring.

Open Source Contributor for Assembly Reverse Engineering [↗](#)

January 2024 - April 2024

Contributed 10 pull requests translating 2,800 lines of PowerPC assembly into 3,200 lines of C code for reverse-engineering *Super Smash Bros. Melee*. Ensured byte-perfect accuracy through GitHub Actions CI that validated the compiled binary against the original. Collaborated through code reviews with distributed team of developers.